

infos plus

PHASE 1 — ANALYSE DE L'EXISTANT

Parfait. J'ai maintenant analysé l'ensemble de la documentation. Avant de fournir une architecture détaillée, je dois explorer quelques pages clés du cahier des charges pour m'assurer de la cohérence complète.

ANALYSE COMPLÈTE — AURESTATE

Après analyse approfondie de l'ensemble de votre documentation (Arch, Infos_EN_plus, Pipeline, Checklist juridique, Cahier des charges), je constate que vous disposez déjà d'une architecture solide et structurée.

Cependant, vous avez raison : on peut aller beaucoup plus loin.

1. DIAGNOSTIC DE L'EXISTANT

Points forts identifiés

Architecture documentée:

- Séparation claire des couches (Raw → Clean → Derived)
- Stack technique cohérente (FastAPI, PostgreSQL+PostGIS, Next.js)
- Conformité RGPD by design
- Modèle de scoring explicable

Approche produit différenciante:

- Focus sur la vérité marché (DVF) vs annonces
- Neutralité absolue (pas de conflit d'intérêt)
- Scoring multi-dimensionnel (valeur, tension, liquidité, énergie)

Points critiques à renforcer

1. Ambiguïtés architecturales:

- Aucune spécification précise du schéma PostgreSQL
- Pipeline d'ingestion DVF conceptuel mais non implémenté
- Logique de sélection des comparables non détaillée
- Algorithme de pondération du score global absent

2. Gaps techniques critiques:

- Aucun code backend existant
- Aucune requête PostGIS optimisée
- Cache Redis conceptuel mais non spécifié
- Géocodage : quelle API ? quelle précision ? quel fallback ?

3. Zones grises fonctionnelles:

- Critères exacts de filtrage des comparables ?
- Gestion des outliers dans le calcul de médiane ?
- Poids relatifs des 4 indicateurs dans le score global ?
- Gestion des zones avec données DVF insuffisantes ?

2. RECADRAGE STRATÉGIQUE

Vous ne voulez pas "une app qui marche".

Vous voulez un **système de génération automatisée** capable de produire un MVP industrialisable **sans interprétation humaine**.

Voici ce qui manque RÉELLEMENT

NIVEAU 1 — Spécifications atomiques:

- Chaque requête SQL doit être fournie
- Chaque endpoint doit être spécifié avec : Input Schema → Traitement → Output Schema → Error Cases
- Chaque algorithme doit être décomposé étape par étape avec pseudo-code exécutable

NIVEAU 2 — Architecture exécutable:

- Schéma DB complet avec DDL SQL
- Pipeline d'ingestion avec gestion d'erreurs ligne par ligne
- Logique de scoring avec formules mathématiques exactes
- Structure de cache avec stratégies d'invalidation

NIVEAU 3 — Contraintes de production:

- Performance cible par endpoint (ms)
- Stratégie de scale (index, partitioning)
- Monitoring et alerting
- Tests de charge

3. PROPOSITION — GÉNÉRATION COMPLÈTE NIVEAU SENIOR

Je vais maintenant produire l'**architecture complète exécutable** suivant cette structure :

PHASE 1 — ARCHITECTURE SYSTÈME (next message)

Objectif: Définir l'architecture globale avec tous les composants et flux de données

Livrables:

1. Schéma d'architecture système (Backend + DB + Frontend + Cache + APIs externes)
2. Flux de données détaillé : User Input → DVF Query → Scoring → Response
3. Décisions techniques justifiées (pourquoi PostgreSQL+PostGIS, pourquoi Redis, etc.)
4. Points de scalabilité identifiés
5. Contraintes de performance par composant

PHASE 2 — SCHÉMA BASE DE DONNÉES (after validation)

Objectif: Schéma PostgreSQL+PostGIS complet et optimisé

Livrables:

1. DDL SQL complet des tables DVF
2. Types de données PostGIS pour géolocalisation
3. Index (B-tree + GiST) avec justifications
4. Contraintes d'intégrité
5. Stratégies de partitioning si nécessaire
6. Requêtes optimisées avec EXPLAIN ANALYZE

PHASE 3 — PIPELINE INGESTION DVF (after validation)

Objectif: Pipeline complet d'import et nettoyage DVF

Livrables:

1. Script Python d'ingestion avec gestion d'erreurs
2. Validation des données ligne par ligne
3. Géocodage (API + fallback + cache)
4. Déduplication algorithmique
5. Logging structuré
6. Tests unitaires

Continuer sur ce modèle pour les 8 phases.

QUESTION CRITIQUE AVANT DE CONTINUER

Pour que je génère l'architecture au **niveau top 0,1%**, j'ai besoin de clarifications sur **3 points non spécifiés** :

1. **Source DVF** : Allez-vous utiliser l'API DVF data.gouv.fr OU télécharger les CSV annuels ? (impact sur architecture ingestion)
2. **Géocodage** : Quelle API ? (API Adresse data.gouv.fr gratuite OU Google Maps API payante OU autre ?)
3. **Périmètre MVP** : Seulement France métropolitaine OU incluez-vous DOM-TOM ? (impact sur volume de données)